

Department of Artificial Intelligence,
Sogang University

Diffusion 기초와 최근 활용

From DDPM to diffusion LLMs

Gwangwoong Park

Business Administration &
Computer Science and Engineering

NLP & ISDS Lab

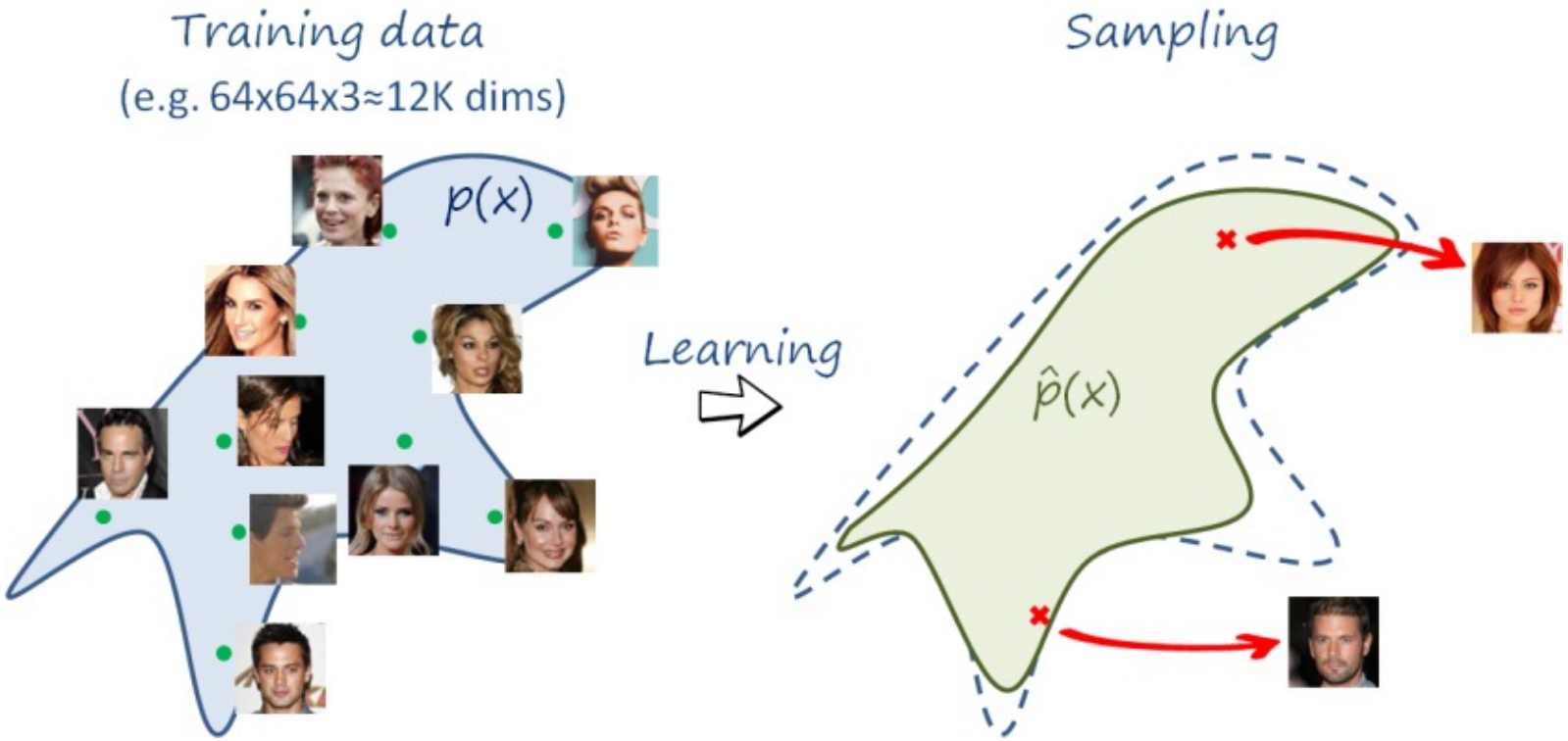
2026.08.05

AGENDA

Agenda

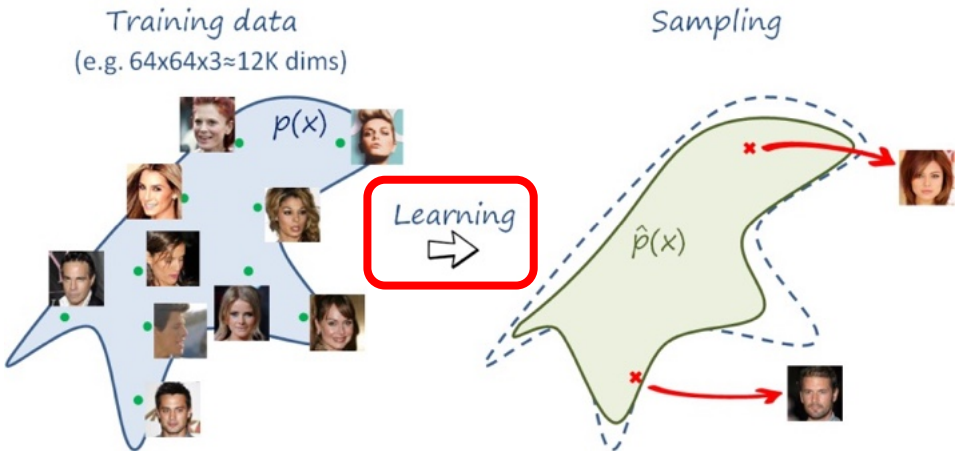
I	생성 모델이란
II	Diffusion 등장 및 원리: DDPM, LDM, DiT
III	Diffusion backbone: Flow matching, MMDiT
IV	활용 1: Video-to-Audio (MMAudio)
V	활용 2: Diffusion LLM (LLaDA)

생성 모델이란: 실제 데이터 분포를 근사하는 분포 모델링



출처: 확률편미분방정식과 인공지능 – 고등과학원 HORIZON.(https://horizon.kias.re.kr/25133/)

생성 모델이란



실제 세상의 데이터들이 특정 확률분포를 따른다고 가정 $p(x)$ → 알 수가 없다 → $p_\theta(x)$를 만들어서 근사해보자

$$\min_\theta D_{\mathrm{KL}}\left(p_{\mathrm{data}} \| p_\theta\right)$$

우리에게 주어진 유한한 데이터셋만 관측 가능 ex) 강아지 이미지들, 얼굴 이미지, 텍스트 말뭉치

관측된 실제 데이터에 높은 확률을 부여하도록 p_θ를 학습한다.

$$\theta^\star = \arg\max_\theta \frac{1}{N}\sum_{n=1}^{N} \log p_\theta(x^{(n)})$$

생성 모델이란

관측된 실제 데이터에 높은 확률을 부여하도록 p_θ를 학습한다.

$$\theta^\star = \arg\max_\theta \frac{1}{N}\sum_{n=1}^{N} \log p_\theta(x^{(n)})$$

확률분포 조건

$$p_\theta(x) \geq 0$$

$$\int p_\theta(x)\,dx = 1$$

정규화 필요

$$p_\theta(x) = \frac{\exp(f_\theta(x))}{Z_\theta}$$

$$Z_\theta = \int \exp(f_\theta(x))\,dx$$

정규화 상수 필요

$$x \in \mathbb{R}^{3\times256\times256}$$

$$Z_\theta = \int_{\mathbb{R}^{196608}} \exp(f_\theta(x))\,dx$$

연산 불가능

$p_\theta(x)$를 정규화된 확률분포로 즉시 모델링하는 것이 어렵다.

다양한 우회 방법론 – 우리가 접하는 방식

모델	핵심 우회 전략	정규화가 가능한 이유	남는 대가
LLM	joint distribution을 조건부 확률의 곱으로 분해	각 token 위치에서의 softmax	Sampling이 순차적
VAE	저차원 Latent z를 도입 z로 부터 x를 생성	Prior와 decoder distribution으로 분해. Gaussian으로 정의	ELBO - 하한 최적화
Diffusion	많은 latent state를 두고 작은 Gaussian transition으로 분할	각 forward/reverse transition을 정규화된 Gaussian으로 정의	ELBO -하한 최적화
GAN	likelihood 접근을 포기하고 두 모델의 minmax 로 대체	$p_\theta(x)$ 값을 계산하지 않는 implicit model	Adversarial training과 mode coverage 문제

Diffusion

- Diffusion 등장 및 원리 – DDPM, LDM, DiT
- Diffusion model backbone – Flow matching, MMDiT

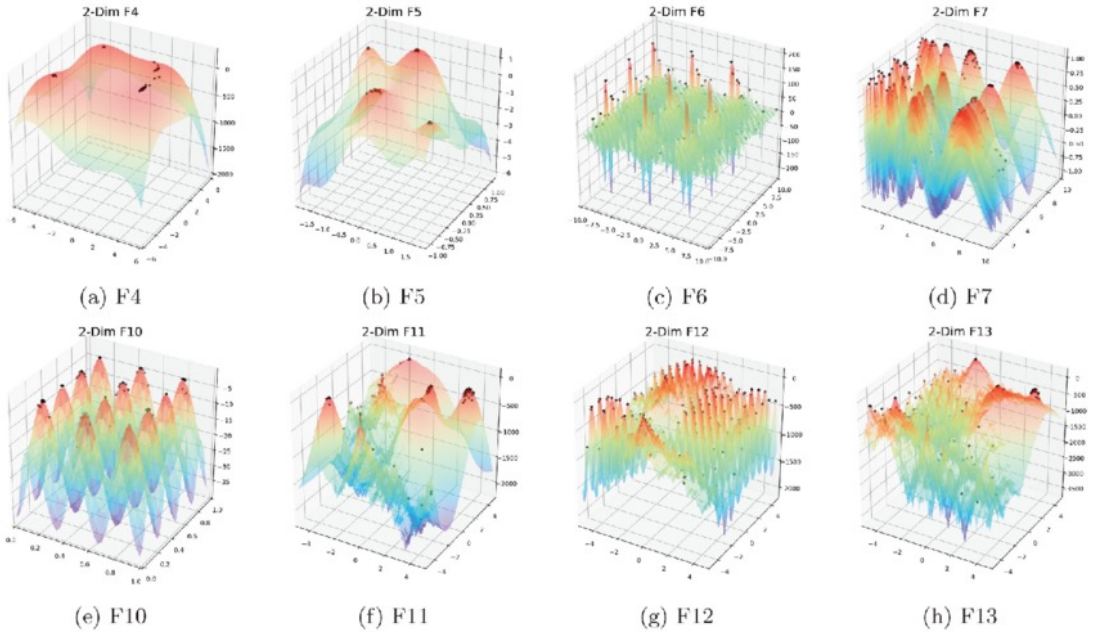
Diffusion의 등장 배경 및 목적은 무엇인가?

유연성 - 다루기 쉬움의 trade-off

머신러닝의 핵심 문제 중 하나는 **매우 유연한(flexible) 확률분포를 통해 복잡한 데이터셋을 모델링**하되,
학습/샘플링/추론/평가가 **해석적 or 계산적으로 다룰 수 있어야 함(tractable)**

- 유연한 분포 — 표현력을 높이면 모델링이 계산상 불가능해짐
- tractable — 간단한 분포로 모델링하면 표현력의 한계

→ 유연하면서도 계산 가능한 분포 $p_\theta(x)$를 어떻게 만들 것인가?



Diffusion의 등장 배경 및 목적은 무엇인가?

물리학에서의 영감

데이터 분포의 구조를 점진적인 forward 확산 과정을 통해 체계적으로 파괴한 뒤, 데이터 구조를 복원하는 reverse 확산 과정을 학습

→ 복잡한 모델링을 매우 쉬운 여러 단계로 쪼개서 유연하고 다루기 쉬운 생성모델을 구현

- 각 단계는 Feller-Anderson theorem에 의해 가우시안으로 모델링 가능 → tractable/easy
- 각 단계를 반복/합성하면 임의의 복잡한 분포를 표현 가능 → Flexible/표현력

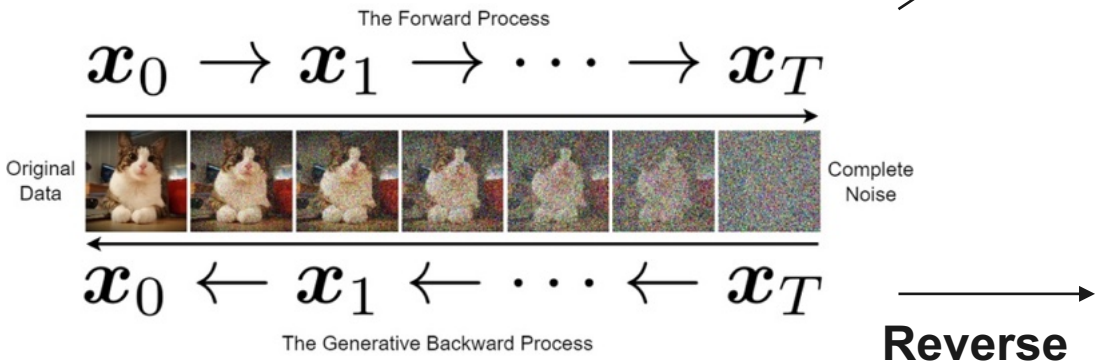
$$\underbrace{p_\theta(x_{t-1}|x_t)}_{\text{단순한 가우시안 하나}} \quad \text{x} \quad T\text{번 반복} \quad = \quad \underbrace{p_\theta(x_0|x_T)}_{\text{복잡하고 유연한 분포}}$$

Forward / Reverse process

Forward

$$x_t = \sqrt{1-\beta_t}x_{t-1} + \sqrt{\beta_t}\epsilon$$

원본 데이터에 가우시안 노이즈를 혼합 반복



Reverse

노이지 데이터를 보고 이전의 원 데이터를 예측

$$p_\theta(x_{t-1}|x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t,t), \Sigma)$$

$\mu_\theta(x_t,t)$를 신경망으로 모델링

$$x_{t-1} = \underbrace{\frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\epsilon_\theta(x_t,t)\right)}_{\mu_\theta(x_t,t)\text{ (한 스텝만큼만 제거)}} + \sqrt{\tilde{\beta}_t}\,z, \qquad z \sim \mathcal{N}(0,I)$$

평균μ_θ, 분산Σ_θ 예측-> 가우시안 분포-> 샘플링을 통해 x_{t-1} 예측 → 수식을 풀면 노이즈 예측으로 단순화 가능

학습, 추론 알고리즘

Algorithm 1 Training

1: **repeat**
2: $\mathbf{x}_0 \sim q(\mathbf{x}_0)$
3: $t \sim \text{Uniform}(\{1, \dots, T\})$
4: $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
5: Take gradient descent step on

$$\nabla_\theta \left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}, t) \right\|^2$$

6: **until** converged

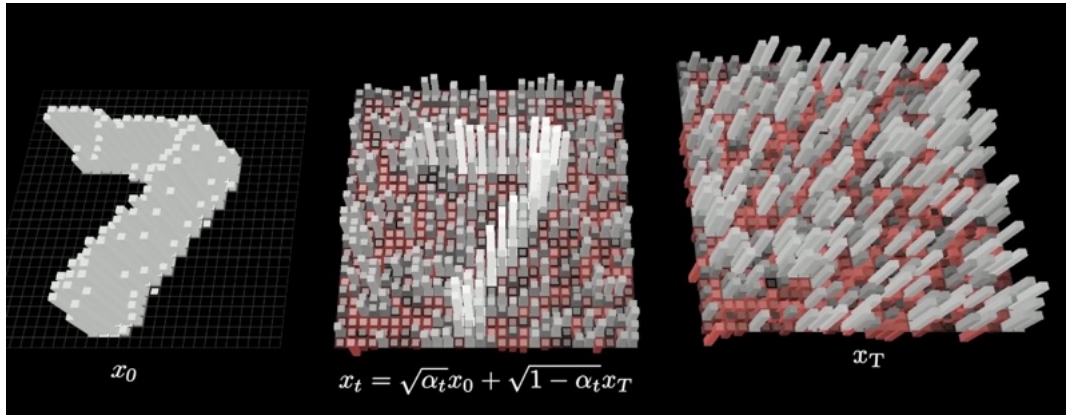
단순한 loss (노이즈 MSE)로 쉽고 안정적 학습

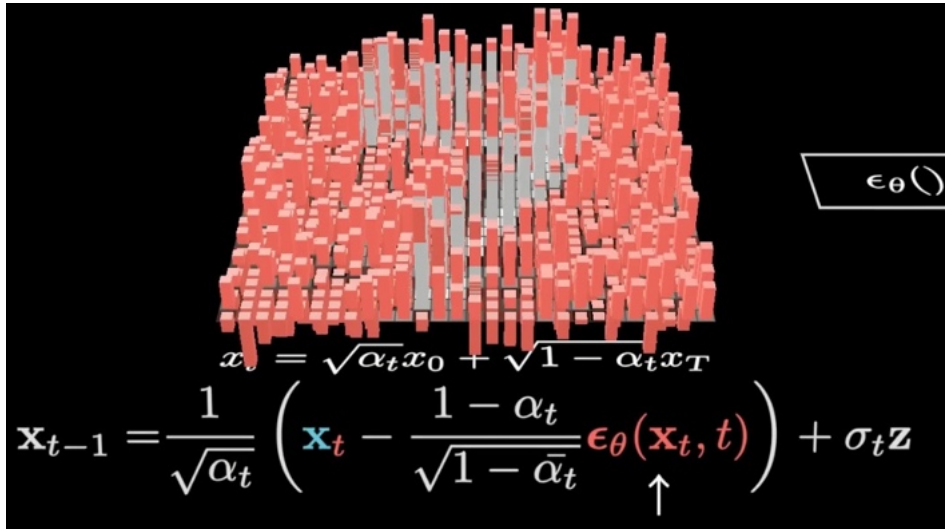
Algorithm 2 Sampling

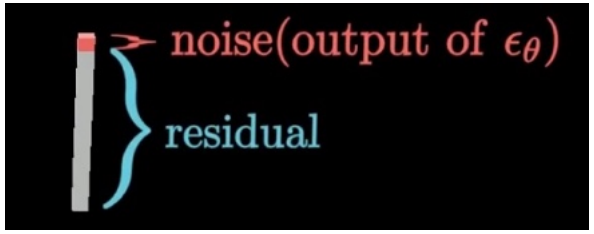
1: $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
2: **for** $t = T, \dots, 1$ **do**
3: $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ if $t > 1$, else $\mathbf{z} = \mathbf{0}$
4: $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}} \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$
5: **end for**
6: **return** $\mathbf{x}_0$

예측한 노이즈를 일부 제거하고 불확실성을 주입 → 조건부 확률분포에서의 샘플링 구현

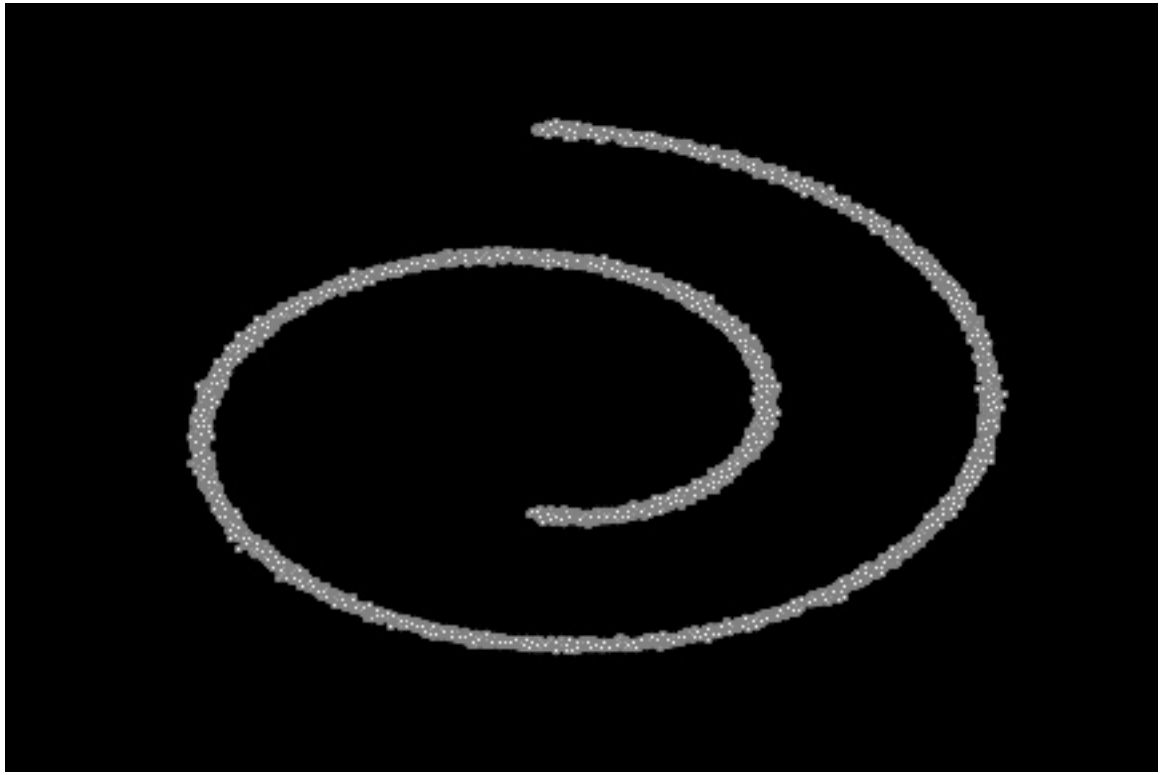
노이즈 예측 및 제거 반복

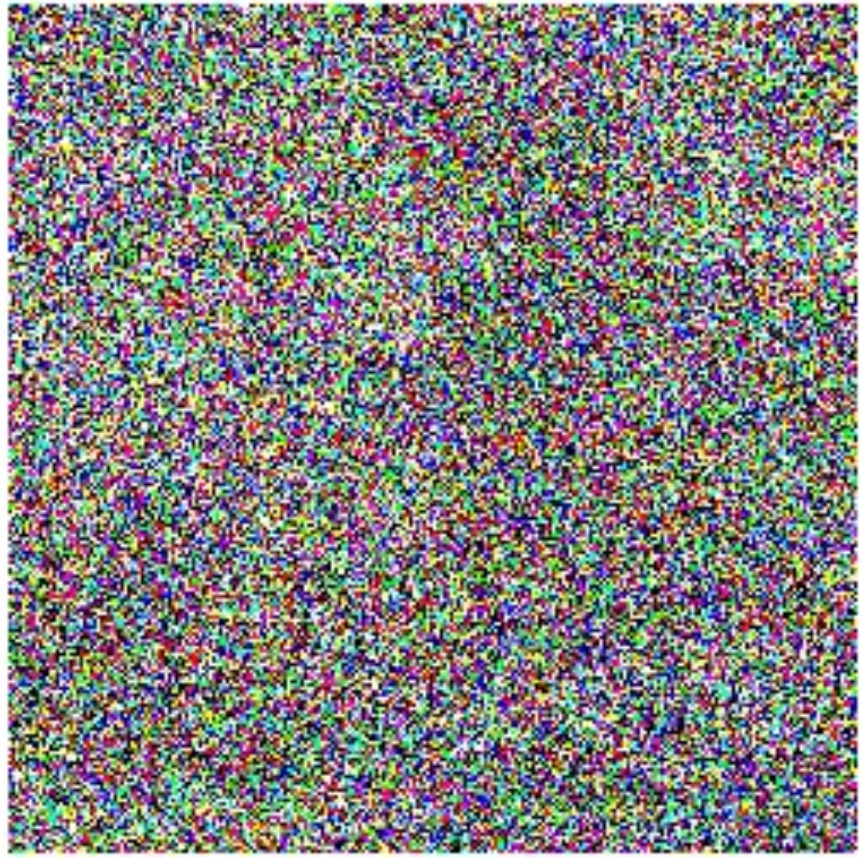

x_0
$x_t = \sqrt{\alpha_t}x_0 + \sqrt{1-\alpha_t}x_T$
x_T


$\epsilon_\theta()$
$x_t = \sqrt{\alpha_t}x_0 + \sqrt{1-\alpha_t}x_T$
$\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}}\left(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\epsilon_\theta(\mathbf{x}_t, t)\right) + \sigma_t \mathbf{z}$


noise(output of ϵ_θ)
residual

노이즈 -> 섞인 노이즈 예측 및 제거 반복 -> 데이터 생성





LDM, DiT

- **LDM(latent diffusion model)**: latent 공간에서의 diffusion process를 통해 픽셀 공간에서의 불필요한 노이즈 예측을 제거
- **DiT(Diffusion Transformer)**: 노이즈 예측 신경망을 Transformer로 모델링, timestep t와 text condition을 adaLN으로 주입하는 아키텍처 제안

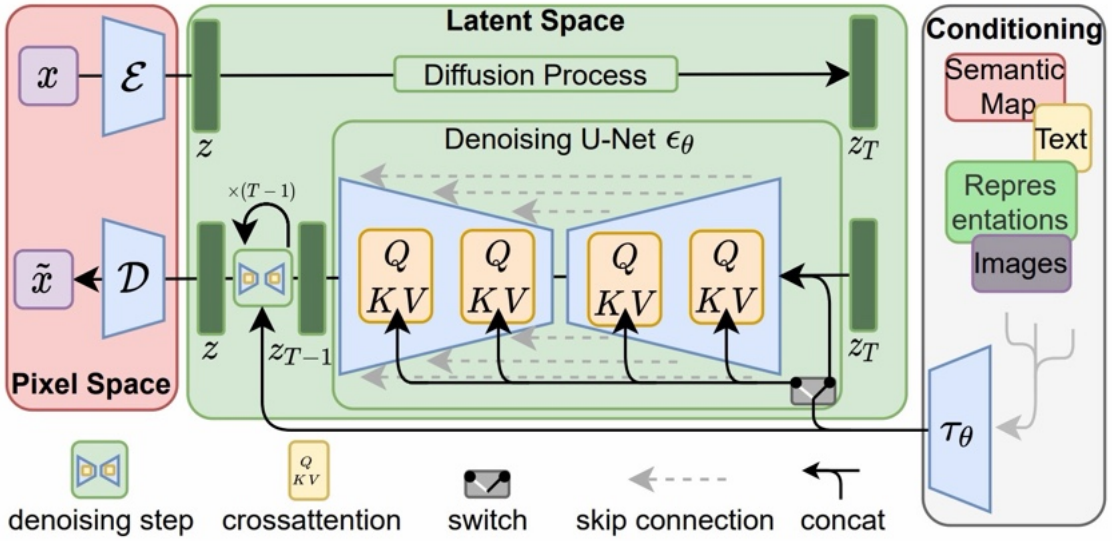
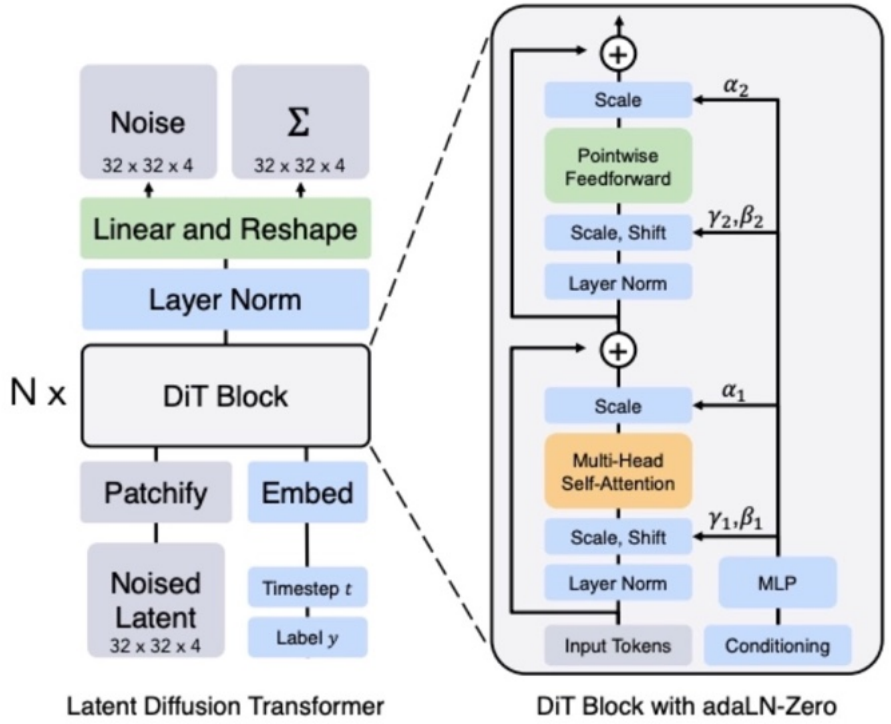


Figure 3. We condition LDMs either via concatenation or by a more general cross-attention mechanism. See Sec. 3.3



Flow matching의 등장

문제의식: Data-noise 경로가 diffusion process에 묶여있다.

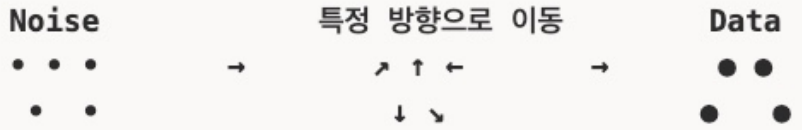
Diffusion은 정해진 noising schedule을 따라 데이터 분포를 Gaussian noise로 변화시킴

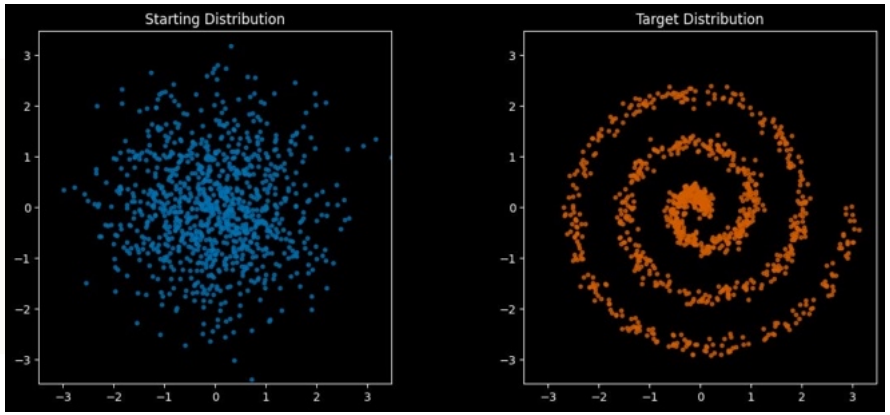
$$p_{\text{data}} \rightarrow p_t \rightarrow p_{\text{noise}}$$

> Diffusion 경로(Gaussian noise를 점차 추가하는 과정)가 유일한 선택일 필요가 있나?
> Data - noise를 더욱 효율적인 경로로 연결할 수 없나? (바로 가면 되지 굳이 Gaussian noising process가 만드는 경로를 따라야하는가)

Flow matching

공간에서 noise 입자들이 data 분포로 이동한다고 생각하자.





이때 공간의 각 위치마다 '지금 이 위치에 있는 입자는 어느 방향으로 얼마나 움직여야 하는가?' 필요

→ **각 원소 값이 얼마나 바뀌어야하는가** (원소별로 음의 방향? 양의 방향? 얼마나 이동? 의 조합)

이를 알려주는 함수가 **velocity field**

$$v_\theta(x, t)$$

- x: 현재 입자의 위치
- t: 현재 시간
- $v_\theta(x, t)$: 현재 위치에서의 이동 방향과 속도

학습 과정

1. 데이터와 noise 한 쌍을 샘플링

$$x_0 \sim p_{\text{data}}$$
$$\epsilon \sim \mathcal{N}(0, I)$$

2. 중간 상태를 직접 정의 — 직선 보간

$$x_t = (1-t)x_0 + t\epsilon$$

$t = 0$	$x_t = x_0$	완전한 데이터
$t = 0.3$	$x_t = 0.7x_0 + 0.3\epsilon$	노이즈 방향으로 이동
$t = 0.8$	$x_t = 0.2x_0 + 0.8\epsilon$	노이즈 방향으로 이동
$t = 1$	$x_t = \epsilon$	완전한 Gaussian noise

3. 중간 상태 식을 t에 대해 미분 → 정답 velocity u_t 계산 (data → noise로 움직이는 속도)

$$\frac{dx_t}{dt} = -x_0 + \epsilon$$
$$u_t = \epsilon - x_0$$

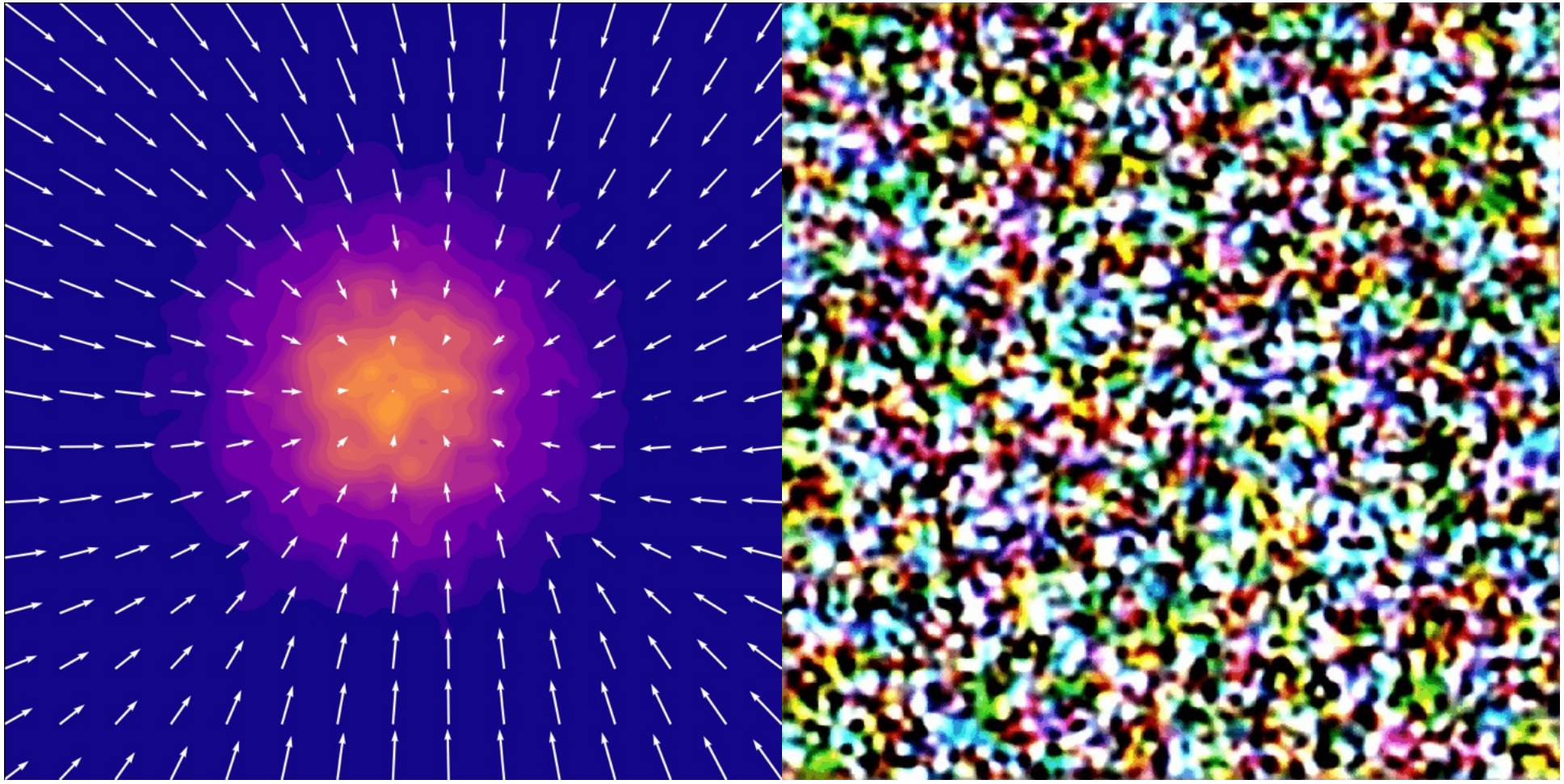
4. 모델은 velocity를 예측

$$v_\theta(x_t, t) \approx \epsilon - x_0$$

5. **Loss**

$$\mathcal{L}_{\text{FM}} = ||v_\theta(x_t, t) - (\epsilon - x_0)||^2$$

노이즈 -> velocity 예측 및 이동 반복 -> 이미지 생성



DiT (Diffusion Transformer)

Transformer의 강세

여러 분야에서 Transformer가 공통 아키텍쳐로 자리잡고 있었음 (NLP, Vision, RL 등)

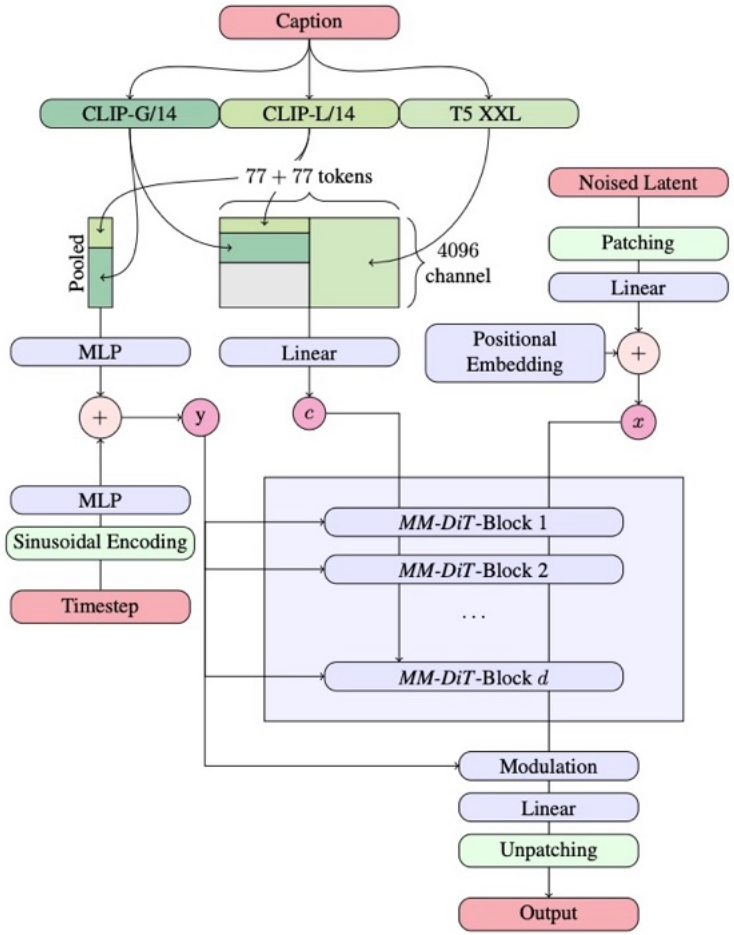
- domain-specific 아키텍쳐를 능가
- 특히 **vision 분야에서 ViT**를 통해 강력한 image recognization model을 구현 + scaling 가능함을 입증

문제의식 — 기존 diffusion architecture는 U-Net으로 고정되어옴

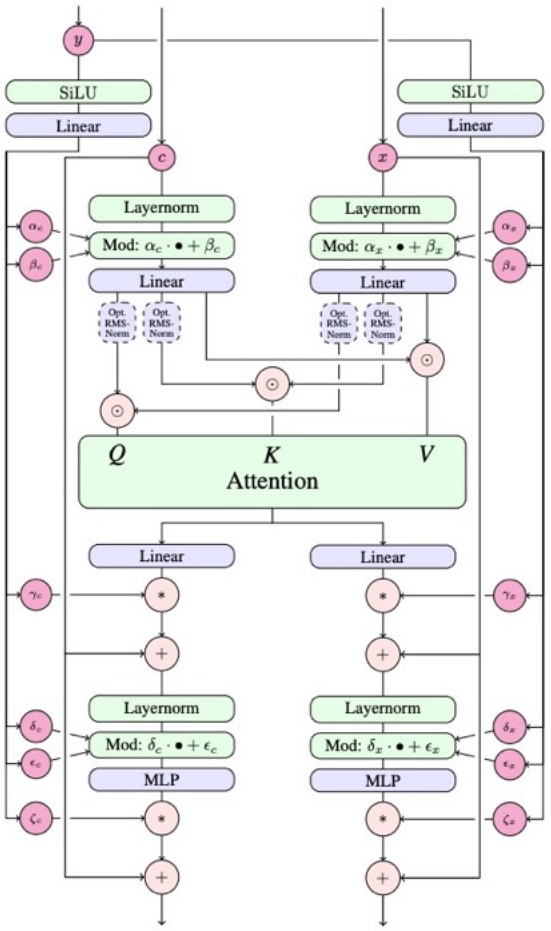
U-Net backbone을 초창기 DDPM 부터 지속적으로 사용해왔음

diffusion 알고리즘, 방법론 등은 빠르게 발전했지만 backbone의 큰 틀은 거의 바뀌지 않았다.

MMDiT



(a) Overview of all components.



(b) One *MM-DiT* block

MM-DiT block

$(c, x, y) \rightarrow (c', x')$

- c: text condition (caption)
- x: image latent
- y: global condition (text + timestep t)

text c, image x에 독립적인 Transformer를 두되 attention 연산에서는 두 sequence를 결합

기호 구분

- $\odot$: sequence concatenation
- $*$: element-wise multiplication
- $+$: residual addition

Figure 2. **Our model architecture.** Concatenation is indicated by $\odot$ and element-wise multiplication by $*$. The RMS-Norm for Q and K can be added to stabilize training runs. Best viewed zoomed in.

Video-to-Audio (MMAudio)

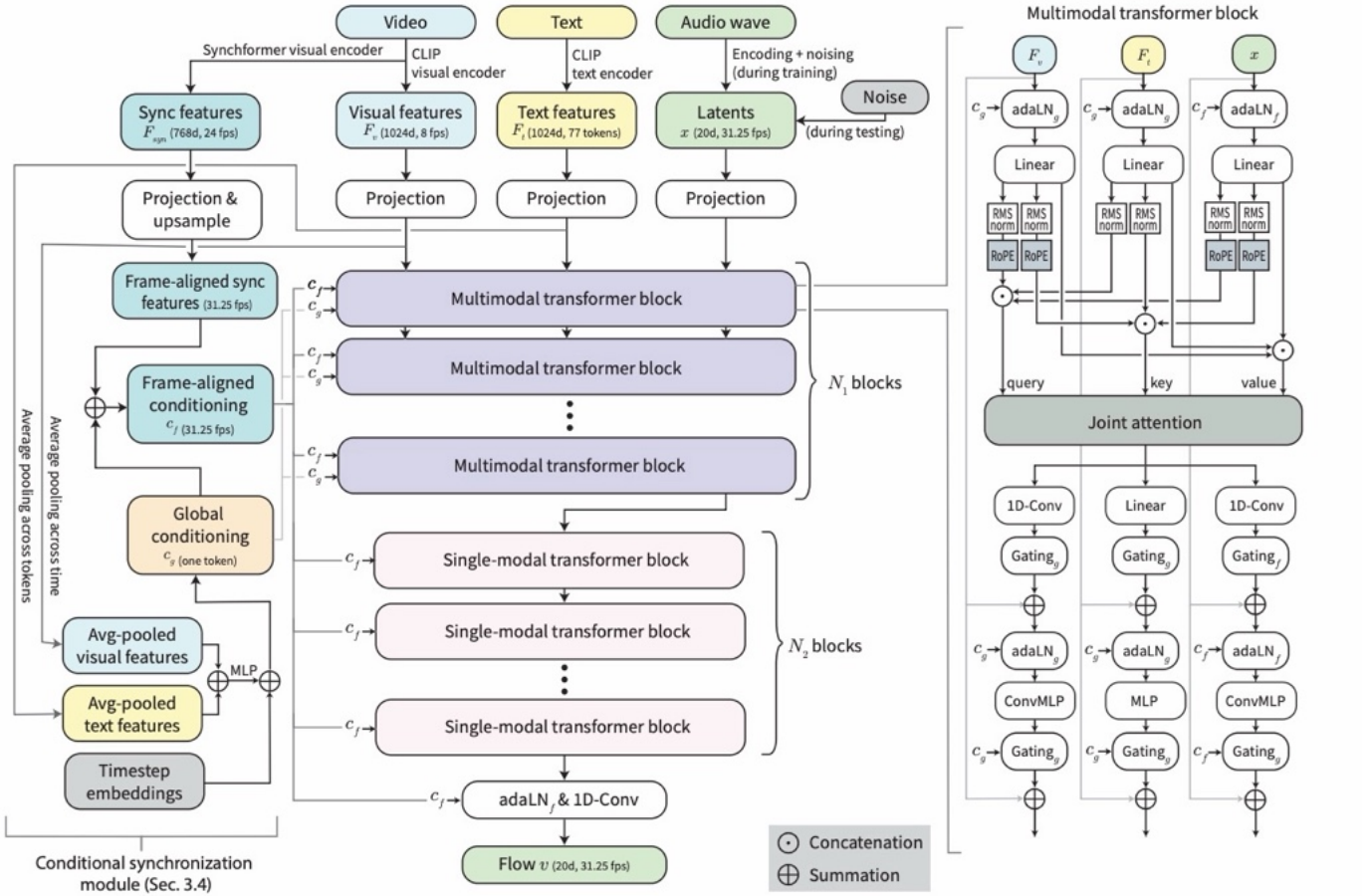


Figure 2. Overview of the MMAudio flow-prediction network. Video conditions, text conditions, and audio latents jointly interact i the multimodal transformer network. A synchronization model (Section 3.4) injects frame-aligned synchronization features for precis audio-visual synchrony.

1. Audio라는 모달리티 추가

- Audio-only DiT blocks
- aligned RoPE

  이미지에서는 2D 공간 정보가 중요했다면, 오디오에서는 실제 시간 정보가 중요

  video - audio간 token rate가 다름 → 정렬을 위해 video stream의 RoPE frequency를 조절 $\frac{31.25}{8}$

$$\text{video stream} : 8 \text{ fps}$$
$$\text{audio latent stream} : 31.25 \text{ fps}$$

- 1D-Conv, convMLP

  국소 정보를 더욱 강조 (주변 frame, audio latent에 더욱 집중하도록) — video, audio 모달리티의 inductive bias 활용

2. Video와의 temporal alignmnet 또한 중요

- Syncformer encoder를 사용한 frame-level synchronization features 생성
- adaLN을 통해 global conditioning을 하되 하나의 vector로 뭉개지 않고, sequence를 보존하여 각 audio latent frame에 맞는 feature를 주입

```
Audio token 1 condition = 전체 장면 정보 + 0.00초 영상 정보
Audio token 2 condition = 전체 장면 정보 + 0.032초 영상 정보
Audio token 3 condition = 전체 장면 정보 + 0.064초 영상 정보
...
```

Diffusion LLM

왜 Diffusion LLM인가? — AR 한계와 발상의 전환

기존 Autoregressive LLM

문장을 왼쪽에서 오른쪽으로 순차 생성 — 한 token을 생성한 뒤, 해당 token을 조건으로 다음 token 생성 반복

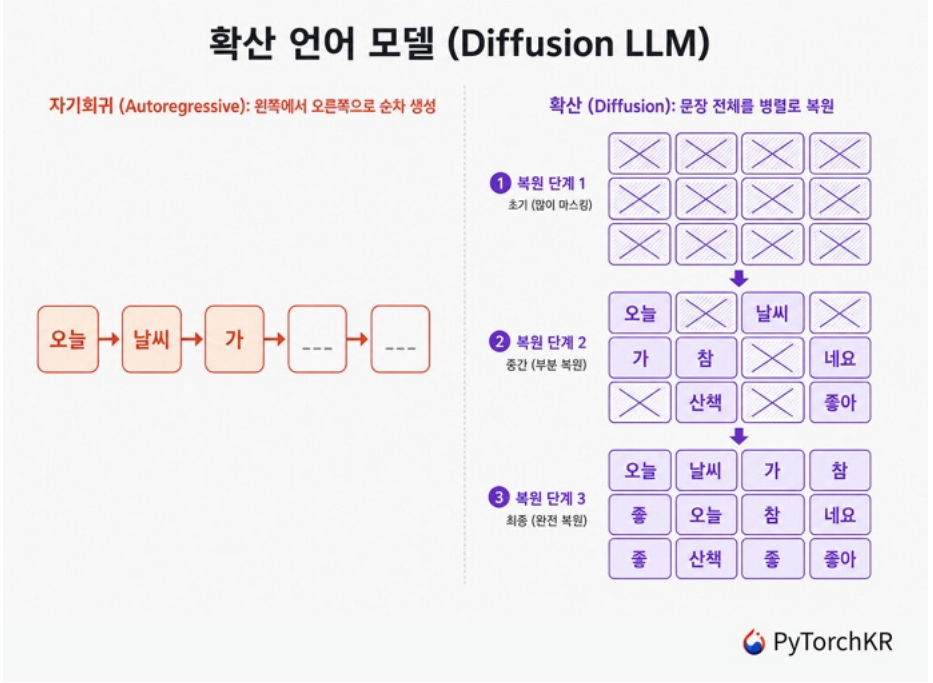
특징

- 생성 순서가 단방향으로 고정
- 한 번 생성한 앞쪽 token을 수정하기 어려움
- 길이에 비례하는 순차적 decoding이 필요 — 토큰 하나를 만들 때마다 모델 전체를 한 번씩 통과해야함
  - 문장이 길어질 수록 지연시간 증가
  - memory-bound로 인한 비효율성 (매번 무거운 가중치를 GPU로 가져와서 연산해야함. 메모리 대역폭으로 인해 '가중치를 가져오는 시간 >> 실제 토큰 생성 연산 시간' → GPU 연산 자원이 남아돌게됨)

DLLM

> 문장을 왼쪽에서 오른쪽으로만 생성하지 않고, 답변 영역 전체를 동시에 예측 및 복원

- 한 번에 여러 토큰을 생성하므로 남는 연산자원을 활용하여 지연시간을 줄일 수 있음 (한 번 가중치를 가져왔을 때, 그 자리에서 여러 개의 토큰을 생성)
- 전체 구조를 잡아가며 확신이 없는 토큰을 순서에 상관없이 재생성 가능



기존 diffusion을 언어모델에 그대로 사용할 수 없는 이유

이미지 latent는 연속값이므로 Gaussian noise를 자연스럽게 더할 수 있음 ex) 1.3 + 0.2 = 1.5

하지만 언어 token은 vocabulary의 정수 index → Gaussian noise를 단순히 더하는 게 불가능

$$\mathrm{ID}(\mathrm{cat}) = 4754$$
$$4754 + 0.2 = ???$$

dLLM 두 가지 접근

Continuous embedding diffusion — Diffusion-LM

> token을 연속적인 embedding vector로 변환한 뒤, embedding space에서 diffusion을 수행하자.

1. 텍스트 토큰을 임베딩을 통해 연속적인 실수 벡터로 변환
2. 실수 벡터에 노이즈를 섞고 복원하는 과정을 학습
3. 추론 시, Gaussian noise를 샘플링하여 복원을 반복
4. 최종 실수 벡터와 가장 가까이에있는 embedding vector에 해당하는 텍스트 token 선택

Masked diffusion — LLaDA

> 이산 token 상태 자체를 오염시키자 — D3PM

- 다른 token으로 무작위 치환
- vocabulary 전체의 uniform 상태로 이동
- **특수한 [MASK] token으로 변경**

Masked Diffusion Model - LLaDA

학습

> 여러 비율로 문장을 가리고, 가려진 모든 token을 동시에 예측

1. token을 가리기

```
t = 0.0
Diffusion language models generate text ## t=0, 모든 토큰이 존재 -> 원본 데이터 상태

t = 0.3
Diffusion [M] models generate [M]

t = 0.7
[M] language [M] [M] [M]

t = 1.0
[M] [M] [M] [M] [M]    ## t=1, 모든 토큰이 마스킹됨 -> 완전한 노이즈 상태 z
```

2. 가려진 토큰을 예측

   Masked sequence 전체를 입력받아서, 모든 masked 위치의 원래 token을 예측

   AR과 달리 casual mask를 사용하지 않고, 왼쪽/오른쪽 token을 모두 보며 예측을 수행

3. 학습목표

   masked 위치에서만 계산하는 cross-entropy

$$\mathcal{L}(\theta) = -\mathbb{E}_{t,x_0,x_t}\left[\frac{1}{t}\sum_{i=1}^{L}\mathbf{1}[x_t^i = M]\log p_\theta(x_0^i \mid x_t)\right]$$

생성

1. user prompt 입력

```
Question: What is 17 × 6?
Answer:
```

2. 응답 영역을 fully masked sequence로 초기화

```
Question: What is 17 × 6?
Answer: [M] [M] [M] [M] [M]
```

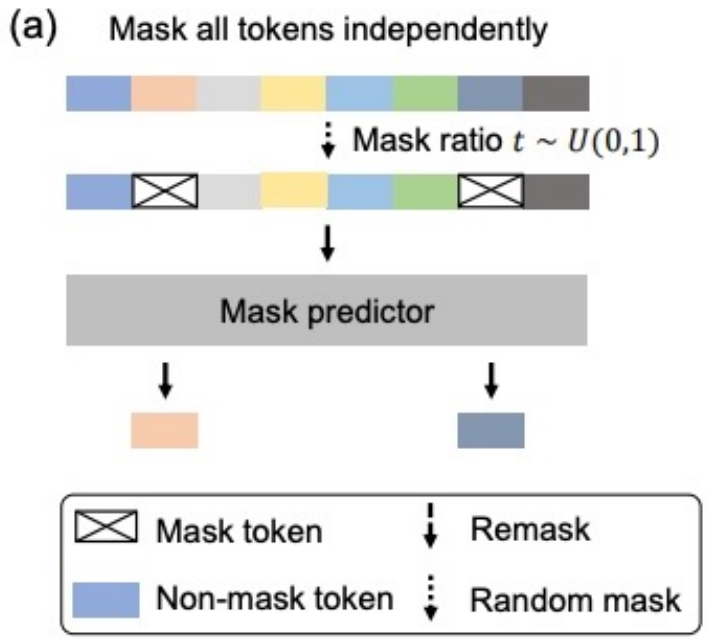
3. sampling을 통해 mask별 token을 생성
   a. 모든 [MASK] 위치의 token 확률을 동시에 예측
   b. 신뢰도가 낮은 위치를 다시 masking / 신뢰도가 높은 위치에서는 token 확정
   c. 모든 mask가 확정될 때 까지 반복

```
Step 0: [M] [M] [M] [M] [M]
Step 1: The [M] is [M] .
Step 2: The answer is [M] .
Step 3: The answer is 102 .
```

LLaDA

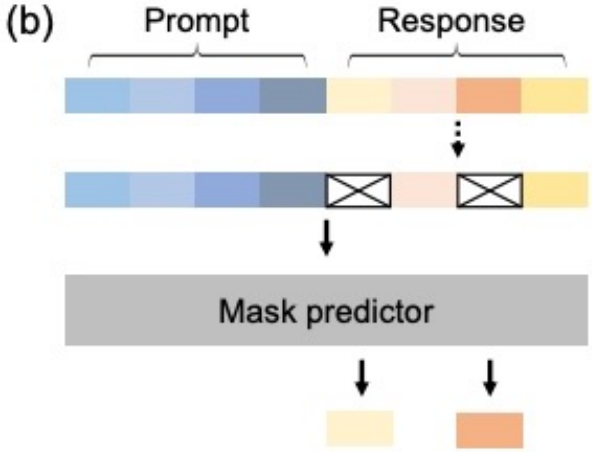
Pre-trainging

전체 문장 masking



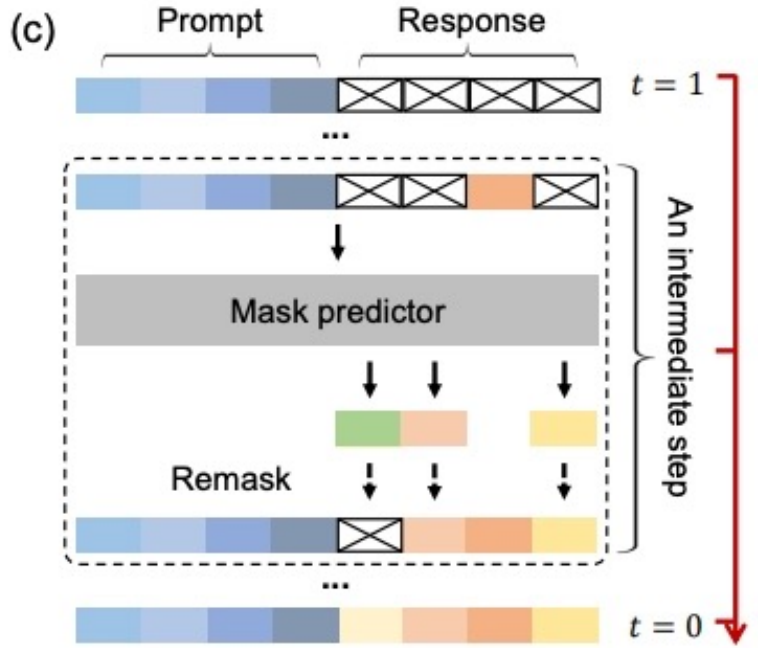
SFT

prompt는 유지, response token에만 masking



Inference

User prompt + Masked sequence → Response



실제 답변 생성 과정

Sampling Process

User	Lily can run 12 kilometers per hour for 4 hours. After that, she runs 6 kilometers per hour. How many kilometers can she run in 8 hours?
LLaDA	

결과 – 동일 사이즈 AR 모델과 경쟁 가능

	LLaDA 8B*	LLaMA3 8B*	LLaMA2 7B*	Qwen2 7B†	Qwen2.5 7B†	Gemma2 9B†	Deepseek 7B¶
Model	Diffusion	AR	AR	AR	AR	AR	AR
Training tokens	2.3T	15T	2T	7T	18T	8T	2T
Post-training	SFT	SFT+RL	SFT+RL	SFT+RL	SFT+RL	SFT+RL	SFT+RL
Alignment pairs	4.5M	-	-	0.5M + -	1M + 0.15M	-	1.5M + -
General Tasks							
MMLU	65.5 (5)	**68.4** (5)	44.1 (5)	-	-	-	49.4 (0)
MMLU-pro	37.0 (0)	**41.9** (0)	4.6 (0)	44.1 (5)	56.3 (5)	52.1 (5)	-
Hellaswag	74.6 (0)	**75.5** (0)	51.5 (0)	-	-	-	68.5 (-)
ARC-C	**88.5** (0)	82.4 (0)	57.3 (0)	-	-	-	49.4 (-)
Mathematics & Science							
GSM8K	**78.6** (4)	78.3 (4)	29.0 (4)	85.7 (0)	91.6 (0)	76.7 (0)	63.0 (0)
Math	26.6 (0)	**29.6** (0)	3.8 (0)	52.9 (0)	75.5 (0)	44.3 (0)	15.8 (0)
GPQA	31.8 (5)	**31.9** (5)	28.4 (5)	34.3 (0)	36.4 (0)	32.8 (0)	-
Code							
HumanEval	47.6 (0)	**59.8** (0)	16.5 (0)	79.9 (0)	84.8 (0)	68.9 (0)	48.2 (-)
MBPP	34.2 (4)	**57.6** (4)	20.6 (4)	67.2 (0)	79.2 (0)	74.9 (0)	35.2 (-)

dLLM 한계와 최신 트렌드

한계

Quality

- AR이 여전히 품질의 기준점
- dLLM은 현재 **일관된 품질 우위**보다 **speed–quality trade-off**를 제공
- DiffusionGemma도 속도 중심의 실험 모델로 제시되고 있는 상황

Systems

- 순차적 복원이 아님. → 표준 AR식 KV cache 적용이 어려움
- 매번 전체 sequence의 반복 계산 비용
- 새로운 최적화 stack 필요 `dLLM-Cache · DyLLM · FlashBlock`

Post-training

- 여러 생성 경로 때문에 sequence likelihood가 다루기 어려움
- AR용 RL을 그대로 적용하기 어려워 별도 목적함수 필요 `diffu-GRPO · VRPO`

최신 트랜드

Block Diffusion

```
Block 1        Block 2        Block 3
[ 병렬 복원 ] → [ 병렬 복원 ] → [ 병렬 복원 ]
       block 사이는 순차적
```

- block 내부: bidirectional·parallel
- block 사이: autoregressive
- 이전 block은 KV cache에 저장
- 병렬성과 variable-length generation을 절충

Diffusion + AR Hybrid

```
Diffusion → 여러 token을 병렬 draft
AR        → draft를 정확하게 verify
```

- Nemotron: 하나의 모델에서 AR·diffusion·self-speculation 지원
- DFlash: 경량 block-diffusion 모델을 speculative drafter로 사용
- Diffusion이 AR을 대체하기보다 **AR의 순차 drafting 병목을 가속하는 부품**으로 활용됨

최신 이슈 – dLLM의 생성 순서

“토큰 생성 순서의 자유는 능력인가, 함정인가?”

JustGRPO

The Flexibility Trap: Rethinking the Value of Arbitrary Order in Diffusion Language Models

🏆 *ICML 2026 Outstanding Paper Award* 🏆

Zanlin Ni[1] Shenzhi Wang[1] Yang Yue[1] Tianyu Yu[2] Weilin Zhao[2] Yeguo Hua[3]

Tianyi Chen[3] Jun Song[4] Cheng Yu[4] Bo Zheng[4] Gao Huang[1]

[1]LeapLab, Tsinghua University [2]NLPLab, Tsinghua University [3]Tsinghua University [4]Alibaba Group

긍정: Plan for the Best

쉬운 위치부터 풀면 어려운 문제를 우회 가능

- 후보가 적은 / 확신이 높은 토큰부터 채우면, 새로운 제약이 생겨서 다음 토큰 예측이 쉬워진다.
- 구조화된 제약 문제에서는 확실한 곳부터 푸는 게 실제 reasoning 전략이 될 수 있다.



반론: The Flexibility Trap

일반적인 reasoning에서는 어려운 token을 미루는 것이 탐색 회피일 수 있다.

- 실제 dLLM sampler는 confidence가 높은 쉬운 token부터 확정하고, 다음과 같은 token을 미루는 경향이 존재
  - Therefore
  - Thus
  - Since
  - Alternatively
- 이러한 token들은 이후 추론 방향을 결정하는 logical fork일 수 있음.
- 미래 결론 내용이 먼저 확정된 뒤에는, 해당 위치가 새로운 추론 경로를 선택하는 분기점이 아니라 이미 쉬운 길로 정해진 결론을 연결하는 빈칸이 될 수 있음.

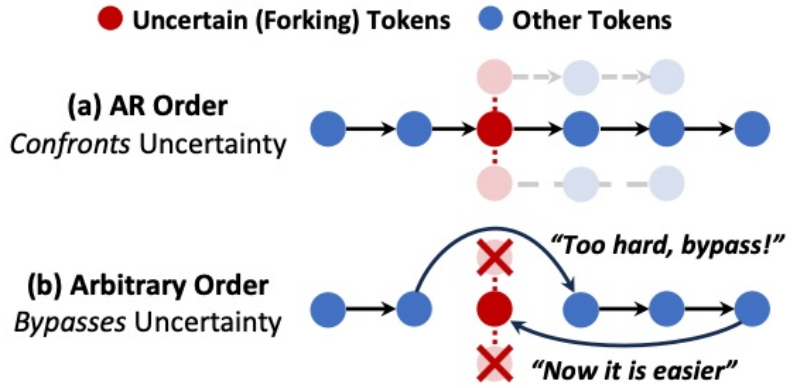


Figure 2: **Confronting *vs.* bypassing uncertainty.** (a) **AR order** preserves reasoning space by forcing decisions at uncertain tokens. (b) **Arbitrary order** bypasses uncertainty and resolves easier tokens first. Once future context is established, the original branching possibilities are effectively pruned.

Pass@k

Pass@k = k회 sampling에서 하나 이상의 정답을 찾을 확률

- ➔ AR 순서에서 pass@k가 더 빠르게 증가
- ➔ AR 순서가 서로 다른 reasoning branch를 더 넓게 탐색한다. | 임의 순서는 여러 번 생성해도 비슷한 풀이 주변에 머묾

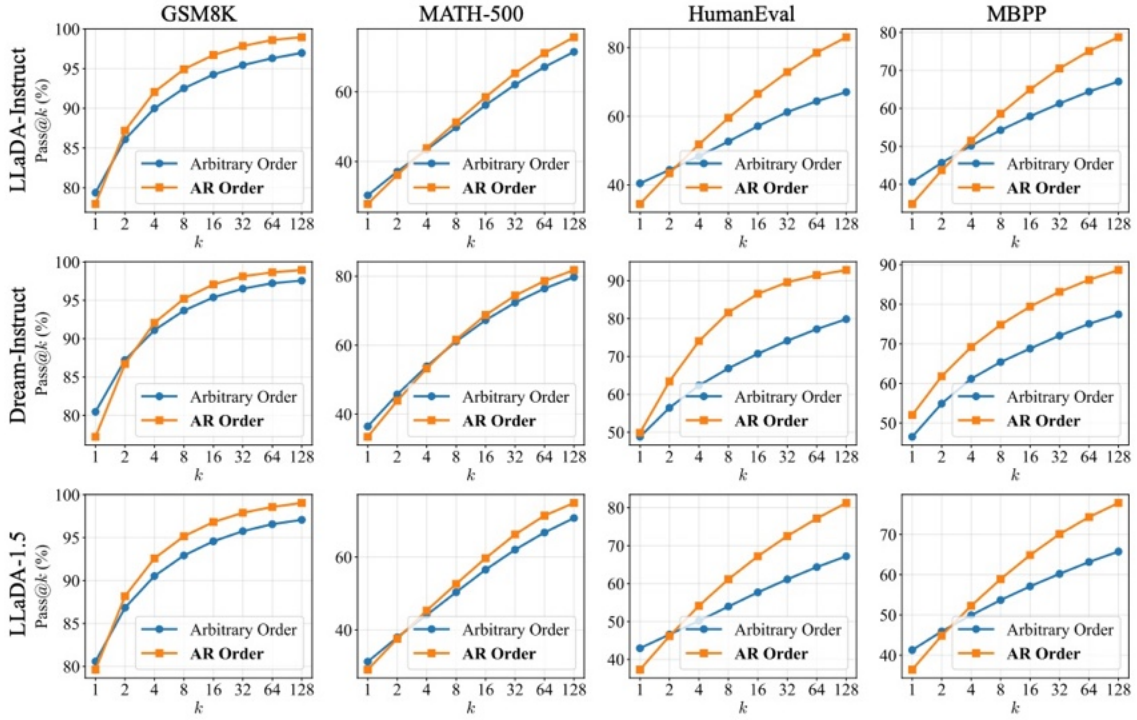


Figure 3: **Reasoning potential measured by Pass@k.** While arbitrary order is competitive in single-shot settings ($k = 1$), it exhibits flatter scaling curves compared to AR Order. This observation is consistent across different dLLMs and general reasoning benchmarks.

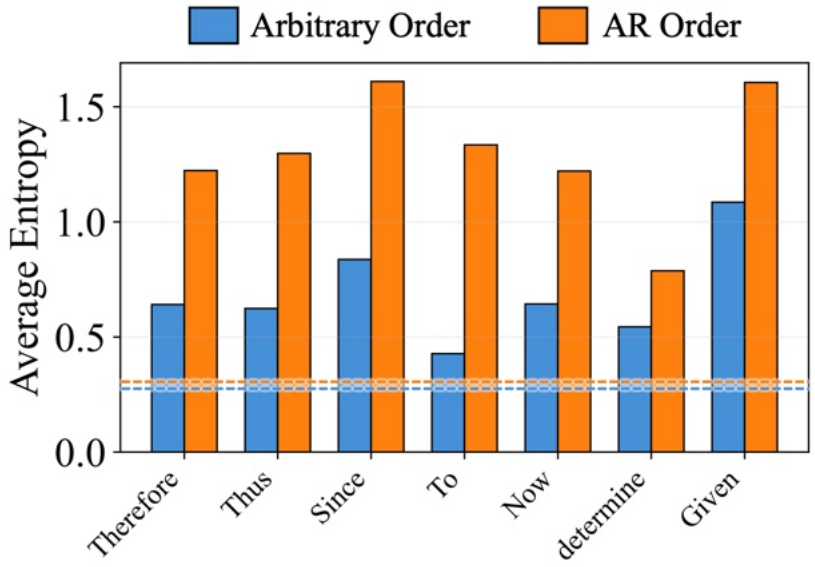


Figure 7: **Entropy degradation.** While the global average token entropy (dashed lines) of arbitrary order remains comparable to AR, the entropy at logical forks (blue bars) drops significantly. Results are measured on MATH-500 with LLaDA-Instruct.

RL 학습의 단순화

임의 순서에서는 같은 답에 도달하는 unmasking 경로가 많음

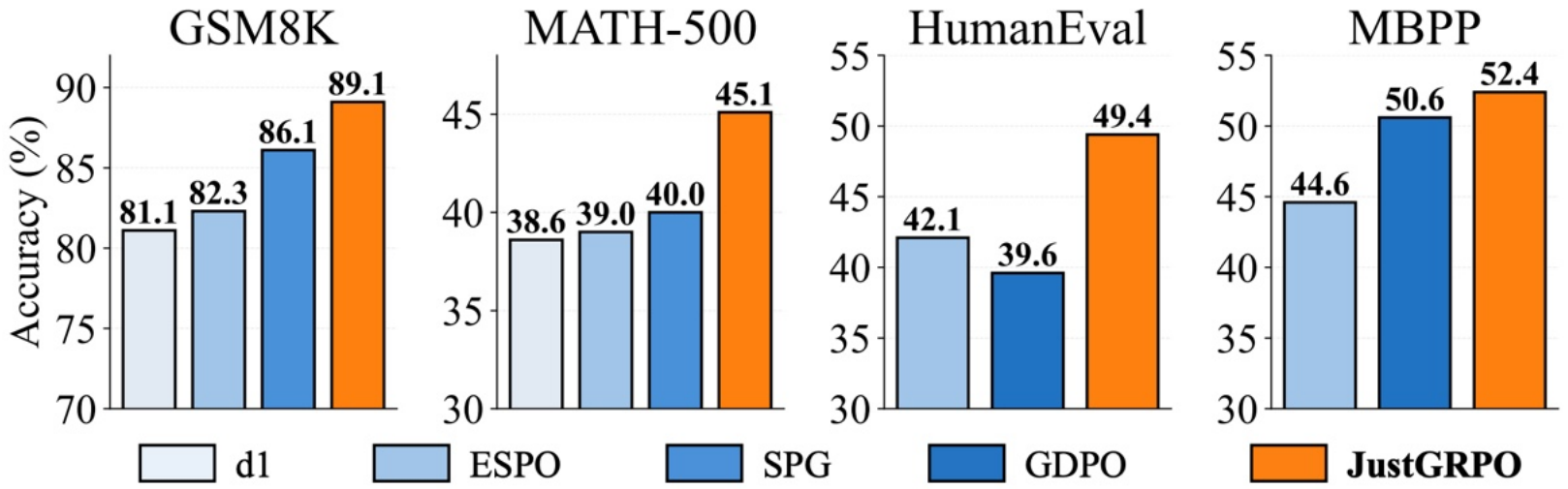
➔ 아래 task가 어려워짐

최종 reward를 어느 denoising 결정에 배치할지

최종 문장의 likelihood 계산

GRPO의 token

반면 AR 순서를 사용하면 기존 GRPO 방법론을 그대로 사용 가능 + 가장 높은 성능 **JustGRPO!**



결론

dLLM의 자유로운 생성 순서가 reasoning에는 항상 이점이 아니다.

- 다양한 풀이 경로를 탐색하기보다는, 분기점을 미뤄 쉬운 경로만 탐색하는 경향 존재
- RL 학습이 복잡해짐

1. RL 학습은 AR 순서대로(왼->오) 수행하여, 탐색을 넓히고 기존 RL 방법론을 활용 가능하게
   학습 후에는 다시 병렬 decoding을 사용하여 dLLM의 장점을 챙기자!